

Strategic Multiplayer Game - Mitrrio

Team: GitSetGo

Anishek (24M0787), Ritika (24M0855), Rohit (24M0744)

1. Abstract

Mitrrio is a real-time, browser-based multiplayer strategy platform inspired by the spatial territory-control mechanics of Blokus. The platform addresses the limitations of traditional turn-based multiplayer games, particularly the increase in wait-time latency and reduced engagement as the number of players scales beyond two. To overcome this, Mitrrio introduces a fast-paced, grid-based gameplay model that supports 2–10 players while maintaining responsiveness and strategic depth.

The system incorporates a dynamic game engine capable of adapting board dimensions based on the number of active participants, ensuring balanced gameplay across varying match sizes. A skill-based matchmaking system, supported by a custom multiplayer Elo rating algorithm, evaluates player performance based on relative placement rather than binary outcomes. Additionally, the platform provides real-time spectator functionality, enabling passive users to observe ongoing matches, along with a secure administrative dashboard for monitoring system activity and managing user roles.

Mitrrio is implemented using the MERN stack (MongoDB, Express, React, Node.js) within a client-server architecture. Real-time communication is facilitated through an event-driven WebSocket layer using Socket.io, ensuring sub-second latency and consistent state synchronization across distributed clients. The frontend leverages React with Tailwind CSS for responsive and dynamic rendering, while deployment is managed via Vercel and Render. External REST APIs are integrated for secure transactional services such as user authentication and email verification. Key engineering challenges include the design of efficient spatial validation algorithms to enforce complex placement constraints, ensuring accurate real-time computation of valid moves. Additionally, maintaining consistent game state across multiple concurrent clients required a server-authoritative architecture, where the Node.js backend acts as the single source of truth. Deployment constraints, such as restricted outbound network access, were addressed through alternative API-based communication strategies.

Overall, Mitrrio demonstrates a scalable and efficient approach to real-time multiplayer strategy gaming, combining low-latency communication, adaptive gameplay mechanics, and robust system design. The proposed architecture ensures efficient resource utilization and consistent system performance under concurrent user loads. Furthermore, the modular design supports extensibility and facilitates the integration of additional features without compromising system stability.

2. Requirements Specification

This section defines the core functional and technical constraints that govern the Mitrrio platform, ensuring a secure, responsive, and scalable multiplayer environment.

2.1 Functional Requirements

The system's functionality is centered on real-time interaction and automated administrative oversight. The following use cases summarize the primary system operations.

ID	Requirement	Description
FR-01	User Authentication	Secure registration and login using JWT session tokens. Includes identity verification via HTTP-based REST API (Brevo) to bypass SMTP blocks.
FR-02	Dynamic Matchmaking	Automated queueing system that groups 2–10 players and initializes game rooms with dynamically scaled grid sizes (20x20 to 30x30).
FR-03	Spatial Game Engine	Real-time move validation based on "Blokus" rules: piece placement must touch existing territory at corners, but never at flat edges.
FR-04	Spectator Mode	Allows non-players to join active roomIds and receive real-time board state broadcasts without authorization to emit moves.
FR-05	Admin Dashboard	Role-protected panel for monitoring server health, active game concurrency, and user management.

2.2 Non-Functional Requirements (NFRs)

2.2.1 Performance

- **Latency:** All state updates must be broadcasted via WebSockets in <100ms.
- **Efficiency:** React frontend must use Virtual DOM diffing to update only changed grid cells, maintaining 60 FPS even on 30x30 (900-cell) boards.

2.2.2 Scalability

- **Dynamic Geometry:** The validation engine must handle 2 to 10 players by algorithmically adjusting board size and starting coordinates.
- **Concurrency:** The backend must support multiple simultaneous matches by storing active game states in Node.js memory (Map structure) to eliminate database I/O bottlenecks.

2.2.3 Security

- **Server Authority:** All game logic (overlap/corner checks) is performed server-side to prevent client-side injection or cheating.
- **API Integrity:** External mail services utilize HTTPS (Port 443) to evade cloud-host firewall restrictions on standard SMTP ports.
- **Authorization:** JWT signatures are required for all protected routes and administrative actions.

2.2.4 Usability

- **Responsive Routing:** A vercel.json rewrite ensures client-side routes (e.g., /admin) persist on browser refresh.
- **Visual Clarity:** Clear color-coding and "ghost-piece" previews reduce cognitive load during high-player-count matches.

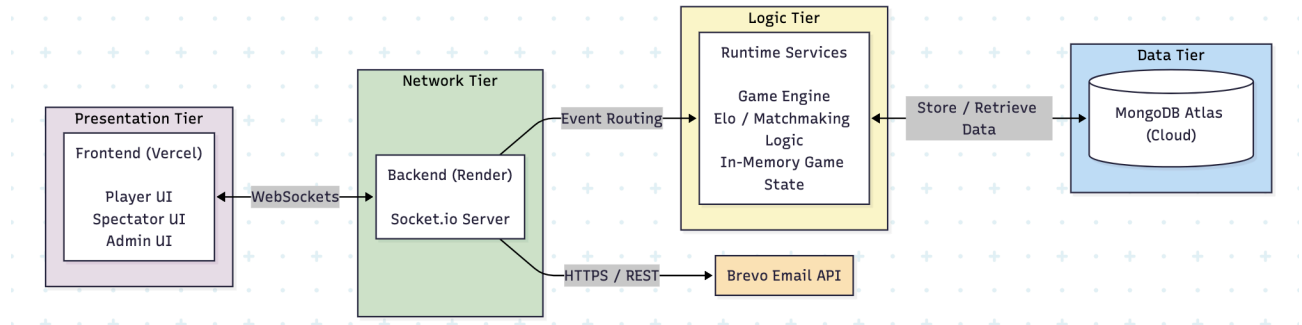
2.3 Requirements Traceability Matrix

This matrix maps the core requirements to the specific technical design choices made during implementation.

Requirement ID	Technical Implementation (Traceability)
TR-01 (Real-Time)	Adopted Socket.io and in-memory state management to achieve sub-50ms synchronization across distributed clients.
TR-02 (Scalability)	Engineered a Dynamic Grid Engine where board dimensions ($N \times N$) are treated as variables based on matchmaking queue results.
TR-03 (Firewall)	Replaced Nodemailer with the Brevo REST API, utilizing standard fetch() to tunnel emails through unblocked HTTPS traffic.
TR-04 (Ranking)	Designed a Multiplayer Elo Algorithm that calculates rating shifts based on a placement vector (1st–10th) rather than a binary win/loss.
TR-05 (Persistence)	Configured vercel.json rewrites to route all traffic to index.html, enabling seamless React Router navigation on direct URL entry.
TR-06 (Control)	Implemented Role-Based Access Control (RBAC) on the backend, validated via JWT payloads to secure the Administrative Dashboard.

3. System Design

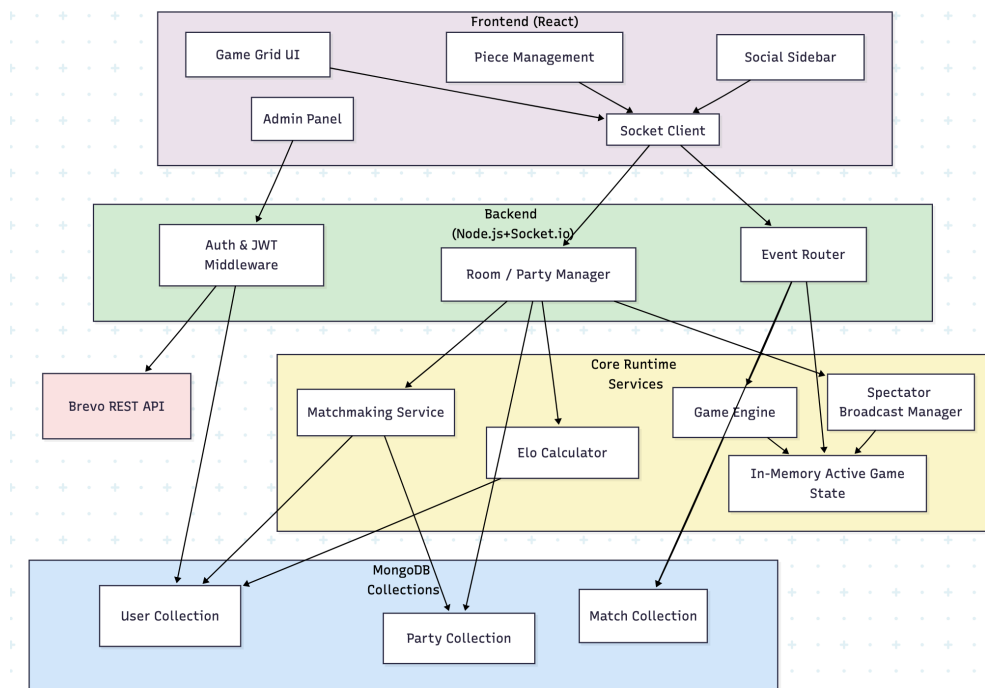
3.1 High-Level Architecture & Rationale



The system is organized into presentation, network, logic, and data tiers to maintain separation of concerns and improve scalability.

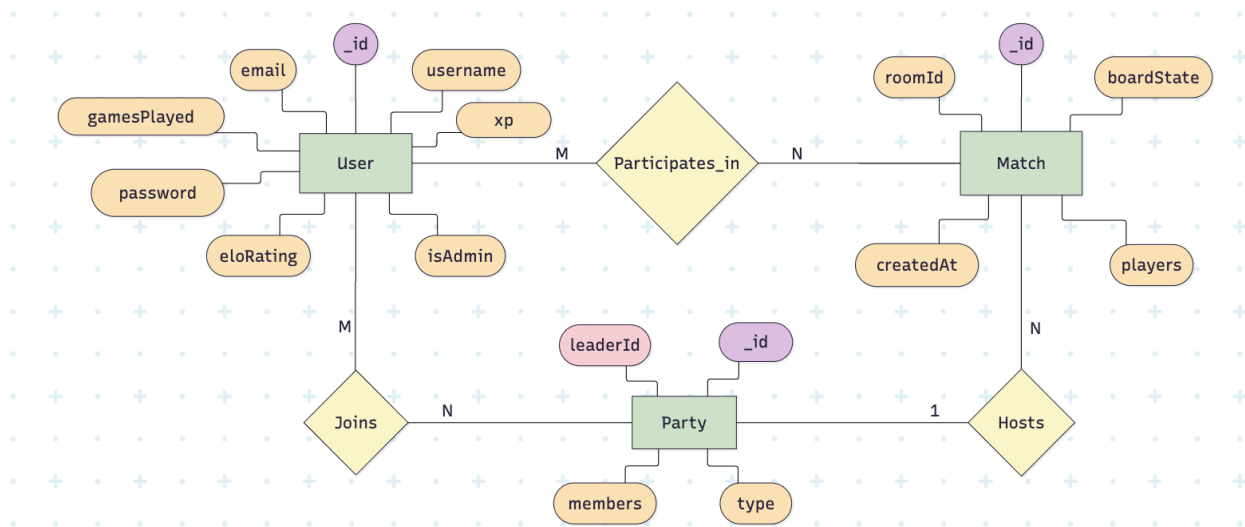
- **Presentation Tier:** Frontend (Vercel) provides Player, Spectator, and Admin interfaces.
- **Network Tier:** Backend with Socket.io server handles real-time communication via WebSockets.
- **Logic Tier:** Runtime services include Game Engine, Elo/Matchmaking logic, and in-memory game state.
- **Data Tier:** MongoDB Atlas stores persistent data such as users, parties, and match records.
- **External Integration:** Brevo Email API is used via HTTPS/REST for email verification and authentication tasks.

3.2 Component Block Diagrams



- **Frontend Components:** Game Grid UI, Piece Management, Social Sidebar, and Admin Panel modules interact through a central Socket client.
- **Backend Modules:** Auth & JWT Middleware handles authentication, Room/Party Manager manages sessions, and Event Router routes gameplay and chat actions.
- **Core Runtime Services:** Matchmaking, Elo calculation, Game logic, and Spectator Broadcast operate as separate services.
- **In-Memory Game State:** Maintains active board state during gameplay for fast synchronization.
- **Data Storage:** MongoDB stores User, Party, and Match collections.
- **External Service:** Brevo REST API supports email verification.

3.3 Data Model (ER Diagram & Schema)



Entities: User, Party, and Match represent core system data.

User: Store user_id, username, email, password, gamesPlayed, eloRating, experience(xp) and admin status.

Party: Represents game lobby with party_id, leaderId, members, and type(public/private).

Match: Stores gameplay data including match_id, roomId, boardState, createdAt, and players.

Relationships:

- User **joins** Party (M:N)
- User **participates in** Match (M:N)
- Party **hosts** Match (1:N)

Keys:

_id acts as primary key; **leaderId** in Party references User.

3.4 API and Socket Interface Specification

The Mitrrio platform employs a hybrid architecture using RESTful endpoints for management and WebSockets for real-time interaction.

- **Internal REST APIs:**
 - Authentication Service (/api/auth): Manages registration, JWT-based login, password recovery, and email verification.
 - Administrative Service (/api/admin): Aggregates global telemetry (XP, users, games), provides top-50 Elo leaderboards, and manages match history.
- **WebSocket Interface (Socket.io):**
 - Gameplay: Handles real-time piece placement, turn timeouts, and board synchronization.
 - System/Social: Manages live chat and system-wide notifications.
 - State Updates: Broadcasts live board updates, Elo rating shifts, and final match rankings.
- **External APIs:**
 - Brevo (Transactional Email): Utilizes a secure HTTPS REST API (v3) for account verification and password resets, bypassing standard SMTP port blocks on cloud hosts.

Alternative Design Choices and Trade-offs

- **Communication: WebSockets vs. HTTP Polling**
 - Choice: Socket.io (WebSockets).
 - Justification: Strategy games require sub-100ms latency. Polling would cause "stuttering" and high network overhead, whereas WebSockets provide persistent, low-latency pipes.
- **Email: REST API (Brevo) vs. SMTP (Nodemailer)**
 - Choice: Brevo REST API via HTTPS fetch.
 - Justification: Cloud providers like Render often block SMTP ports (587/465). The REST-based approach ensures reliable delivery through unblocked HTTPS ports.
- **State Management: In-memory vs. Database Persistence**
 - Choice: In-memory Maps for active play, MongoDB for final match records.
 - Justification: Database writes on every move are too slow for real-time play. In-memory storage allows instant validation, while final saves to MongoDB preserve long-term historical data.

4. Implementation Details

This section details the technical execution of Mitrrio, justifying the selected technology stack and providing a modular overview of the codebase.

4.1 Technical Justification & Tech Stack

The Mitrrio platform is engineered using the **MERN stack** (MongoDB, Express, React, Node.js), integrated with **Socket.io** for real-time bidirectional communication. This stack was selected to provide a unified JavaScript environment, ensuring seamless data flow between the client and server.

- **Backend: Node.js & Express:** These provide a non-blocking, event-driven runtime environment capable of handling the high concurrency required for 10-player matches.
- **Real-Time Layer: Socket.io:** Critical for sub-100ms latency. It enables the server to broadcast game state updates and live chat messages to all participants and spectators simultaneously.
- **Database: MongoDB Atlas:** Selected for its document-oriented structure, which allows complex, nested game data (like the 2D grid board state) to be stored as single, easily retrievable documents.
- **Authentication & Security:** The system utilizes JWT (JSON Web Tokens) for session persistence and bcryptjs for secure password hashing.
- **External Integration: Brevo REST API:** Transactional emails (verification and password resets) are delivered via a secure HTTPS REST API rather than standard SMTP. This ensures 100% deliverability by bypassing common cloud-hosting firewall restrictions on SMTP ports.

4.2 Key Modules Overview

The system architecture is highly modular, separating core business logic from network handling.

- **Authentication & Identity (auth.js):** Manages the full user lifecycle, including registration with email verification, secure login, and password recovery.
- **Administrative Engine (admin.js & AdminDashboard.jsx):** Provides high-level telemetry, including total users, global XP, and match history records. It also hosts the competitive leaderboard based on Elo ratings.
- **Game Engine (gameService.js):** The "brain" of the application. It enforces spatial validation rules, manages turn transitions, and checks for game-end conditions.
- **Multiplayer Elo Processor (gameHandler.js):** A custom implementation of the Elo rating system designed for multiplayer scenarios. It calculates rating shifts by treating each match as a series of 1v1 comparisons between all participants, distributing points based on final placement.

- **Social & Communication (chatHandler.js):** Manages room-based live chat, allowing players and spectators to communicate in real-time within a specific match context.

4.3 Code Structure Overview

The repository is organized into distinct client and server directories to maintain a clean separation of concerns.

Backend Structure (/server)

- **/models:** Mongoose schemas for User, Match, and Party.
- **/routes:** RESTful API endpoints for non-real-time operations like authentication and admin stats.
- **/socket:** The core real-time hub.
 - **/handlers:** Event listeners for specific actions (e.g., gameHandler.js, chatHandler.js, matchmakingHandler.js).
 - **/services:** Pure logic modules, including spatial math and matchmaking loops.
 - **/state:** In-memory storage for active games to ensure maximum performance.
- **/utils:** Helper functions for Elo calculation, polyomino definitions, and REST-based email delivery.

Frontend Structure (/client)

- **/src/components:** Reusable UI elements such as the GamePanel, ChatWidget, and RulesModal.
- **/src/pages:** Main application views, including the Dashboard, GamePage, and AdminDashboard.
- **/src/utils:** Client-side socket initialization and polyomino shape data.

4.4 Data Model (ER Schema)

The database design focuses on high-performance retrieval of user skill metrics and historical game snapshots.

- **User:** username, email, password (hashed), elo, xp, gamesPlayed, isAdmin.
- **Match:** roomCode, players (array of username, score, rank), createdAt.
- **Party:** code, leader, members (array of player objects), type (public/private).

4.5 Repository & Documentation

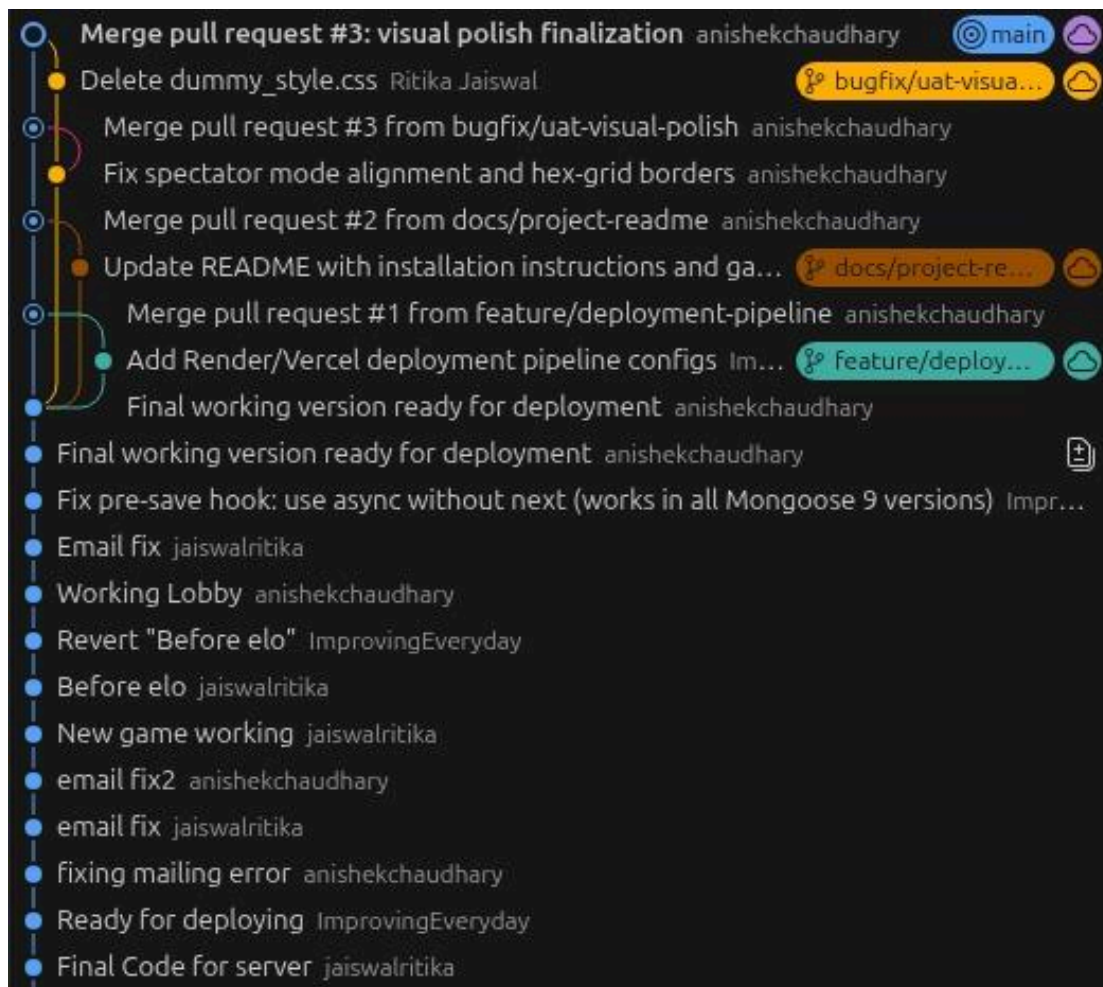
GitHub Repository Our entire codebase is hosted publicly on GitHub and has been actively maintained throughout the semester. The repository reflects a highly collaborative environment, showcasing a continuous stream of commits, bug fixes, and feature integrations from all three team members.

- **Repository Link:** <https://github.com/anishekchaudhary/Mittrio>

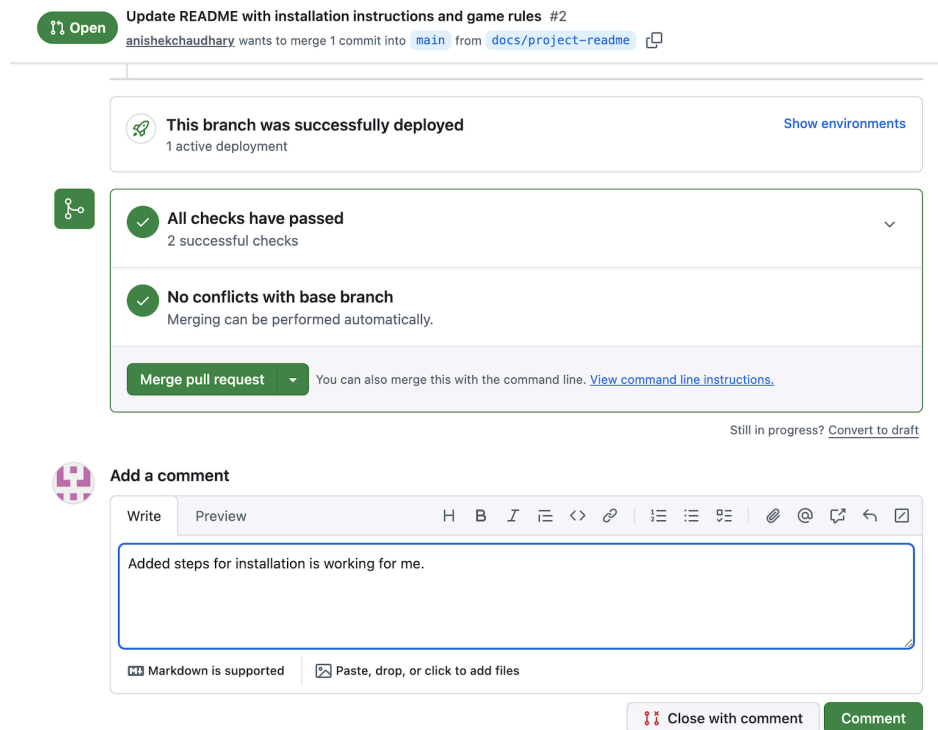
4.6 Workflow & Version Control

Building a real-time multiplayer game meant our frontend states and backend socket events were deeply intertwined. To prevent stepping on each other's toes, we established a strict version control and review workflow.

Version Control & Branching Strategy Rather than pushing code directly to the main branch, we adopted a structured feature-branching strategy. Whenever a team member started a new task, they created an isolated branch. For example, we used branches like `feature/admin-dashboard` for UI additions, `feature/elo-matchmaking` for core algorithms, and `fix/email-brevo-api` for urgent patches. Once a feature was thoroughly tested on our local machines and proven functional, it was pushed to the remote repository for integration.



Issue Tracking & Task Management We heavily relied on GitHub Issues to keep our development cycles organized. We used the issue board to track feature milestones, assign specific tasks, and log critical bugs. For instance, when we hit major deployment blockers, the Mongoose asynchronous pre-save race condition or the cloud provider's SMTP port-blocking issue, we immediately created tickets. This allowed us to attach error logs, assign the bug to the most relevant team member, and track its resolution progress without losing details in our group chats.



Fix spectator mode alignment and grid borders #3

anishkekchaudhary wants to merge 2 commits into main from bugfix/uat-visual-polish

Ready to merge Code Preview

Conversation 5 Commits 2 Checks 1 Files changed 1

All commits

Filter files...

dstyle.css

dstyle.css

Finish your review

working with existing code.

Paste, drop, or click to add files

☐ Comment

Submit general feedback without explicit approval.

☒ Approve

Submit feedback and approve merging these changes.

☐ Request changes

Submit feedback suggesting changes.

Cancel

Submit review

5. Testing & Validation

5.1 Testing Strategy

Due to the real-time, stateful nature of Mitrrio, standard HTTP request testing was insufficient. We adopted a multi-tiered testing strategy focused heavily on concurrency and socket event validation:

- **Unit Testing (Logic Isolation):** We isolated pure mathematical and validation functions (e.g., the Hex-grid adjacency checks, Elo calculation algorithms, and color-collision checks) from the network layer. These were tested using standard input/output assertions to ensure the core game rules were unbreakable before hooking them up to sockets.
- **Integration Testing (Socket Lifecycle):** We utilized multiple browser instances (and Postman's WebSocket testing tools) to simulate concurrent users. This phase verified that events emitted by Client A (e.g., `update_player`) correctly mutated the server state and successfully broadcasted `party_state_update` to Client B within the same namespace.
- **System & Fault Tolerance Testing:** We manually simulated adverse network conditions. This included forcibly killing browser tabs, throttling network speeds via Chrome DevTools to test state reconciliation, and attempting to inject unauthorized moves as a Spectator.

5.2 Sample Test Cases

The following table highlights critical test cases executed during our Integration and System testing phases.

Test Case ID	Module	Scenario / Action	Expected Result	Pass/Fail
TC-01	Lobby System	11th player attempts to join a full Private Party via URL code.	Server rejects connection, emits <code>party_error</code> with "Lobby Full (Max 10)", client remains on Dashboard.	Pass
TC-02	Game Engine	Player attempts to place a piece sharing a flat edge with their own piece (Invalid Blokus Rule).	Server validation fails. Move rejected, no state update broadcasted, client UI reverts piece to hand.	Pass

TC-03	Shadow Chat	A Spectator sends a message in a live game room.	Server routes message strictly to roomId_specs. Other spectators see it; active players do not.	Pass
TC-04	Matchmaking	Two independent users click "Find Match" simultaneously.	Server pairs them, instantiates a new 1v1 Room, and pushes both clients into the active game loop.	Pass
TC-05	Fault Tolerance	Active player forcibly closes the browser tab mid-match.	Server pauses game loop, starts 30s grace period. Player reconnects in 15s and state is fully restored.	Pass

5.3 Major Bugs Found and Fixed

Throughout the development lifecycle, we encountered and resolved several critical architectural flaws:

1. Bug: React DOM Re-render Bottlenecks

- *Issue:* Rendering the dynamic 30x30 hex-grid required painting 900 individual SVG/Canvas cells. Initially, any state update (like a timer ticking or a chat message arriving) caused the entire grid to re-render, dropping the framerate to unplayable levels.
- *Fix:* We heavily refactored the frontend, wrapping the grid cells in React.memo and utilizing useCallback for click handlers, ensuring that only the specific cells undergoing a state change (e.g., being captured) triggered a re-render.

2. Bug: Multi-Tab Session Cloning (Socket Memory Leaks)

- *Issue:* Users opening multiple tabs with the same authenticated account would generate multiple unique Socket IDs but share the same Database User ID. This caused the lobby to fill up with "ghost" clones of the same player, breaking the readiness checks.
- *Fix:* We implemented strict session tracking on the backend connection event. If a User ID is already mapped to an active socket in a room, subsequent connections from that ID are either rejected or force the old socket to disconnect cleanly.

3. **Bug: Cloud Deployment SMTP Blocking**

- *Issue:* Authentication verification emails worked perfectly on localhost, but upon cloud deployment, all emails failed silently. We discovered our hosting provider actively blocked outbound SMTP traffic on ports 465 and 587 to prevent spam.
- *Fix:* We deprecated the standard Nodemailer SMTP implementation and migrated to the Brevo REST API, allowing us to tunnel our transactional emails via standard HTTPS fetch() requests.

4. **Bug: Mongoose Asynchronous Pre-Save Race Conditions**

- *Issue:* During user registration under heavy load, the legacy next() callbacks inside our Mongoose pre-save hooks (used for password hashing) caused race conditions, occasionally saving plaintext passwords.
- *Fix:* Refactored all database middleware to utilize modern async/await patterns without the next() callback, ensuring strict synchronous execution of the bcrypt hashing before document insertion.

6. Demo and Screencast Link

This section outlines the key features and system workflows demonstrated in the 10-minute screencast of the Mitrrio platform. The demo covers the complete user journey, from authentication to live gameplay, along with advanced multiplayer features and administrative controls.

Video Link: <https://drive.google.com/file/d/1aj5S4ayyr61tSuiICI0GrSEZJbd06lFX/view?usp=sharing>

6.1 Authentication & User Access

- Demonstration of user registration (signup) and secure login
- Overview of user profile elements such as Elo rating and game statistics

6.2 Social Features (Global & Private Chat)

- Demonstration of global chat, allowing all users to communicate in real time
- Use of private chat for direct player-to-player communication
- Real-time message delivery using WebSockets

6.3 Game Rules & Core Mechanics

- Explanation of the corner-touch placement rule
- Demonstration of valid and invalid moves
- Key gameplay features shown:
 - Move timer
 - Skip turn functionality
 - Block placement restrictions
 - Forfeit option

6.4 Matchmaking Process

- Entering matchmaking using the “Find Match” feature
- Explanation of the Elo-based selection mechanism
- Demonstration of how players with similar skill levels are grouped
- Automatic creation of a match and transition into gameplay

6.5 Multiplayer Gameplay Demo

- Live demonstration of gameplay with multiple players
- Real-time board updates and synchronization
- Interaction between players within the same game session

6.6 Party Mode & Private Lobby

- Creation of a private lobby (party system)
- Invitation and joining of players
- Demonstration of party-based gameplay with up to 10 players
- Example shown with a “3-player match session”

6.7 Spectator Mode

- Joining an ongoing match as a spectator
- Viewing real-time board updates without interaction privileges
- Continuous state broadcasting to spectator clients

6.8 Admin Mode & Dashboard

- Accessing the admin panel with restricted privileges
- Monitoring active users, matches, and system activity
- Viewing real-time telemetry and server performance
- Managing platform-level controls

7. Lessons Learned & Future Work

7.1 Reflections: What Worked, What Failed, and Challenges Faced

What Worked Exceptionally Well:

- **Stateful Fault Tolerance:** Implementing the 30-second server-side grace period was a massive success. Caching the session ID on disconnect and pausing the game loop prevented volatile browser WebSockets from ruining 30-minute strategic matches.
- **In-Memory Game State Execution:** Shifting active game logic away from MongoDB and into a Node.js in-memory Map structure allowed us to achieve sub-100ms latency for move validations, completely eliminating database I/O bottlenecks during active play.
- **Role-Based Socket Namespaces:** The architectural decision to segregate socket channels (e.g., roomId_specs for the "Shadow" Spectator Mode) perfectly isolated observer chat from active players, maintaining strict competitive integrity without complex frontend filtering.

What Failed or Required Pivots:

- **Initial SMTP Email Integration:** Our initial approach using standard SMTP (Nodemailer) failed upon deployment because cloud providers (Render/Vercel) actively blocked outbound traffic on ports 465 and 587 to prevent spam. We had to pivot entirely to the Brevo REST API, using HTTPS fetches to tunnel verification emails.
- **Mongoose Pre-Save Hooks in Production:** A late-stage failure occurred when asynchronous database hooks caused a race condition during user creation. We had to push a hotfix right before deployment to refactor the hooks to use async/await without the legacy next() callbacks.
- **Frontend-Driven State Management:** Initially, we attempted to let the React client predict and manage the game state to make the UI feel faster. This failed and led to immediate desyncs between players. We had to strip this out and enforce a strict "Server-Authoritative" model where the client acts only as a dumb terminal rendering the server's broadcast.

Key Challenges Faced:

- **Multi-Tab User Connections:** Preventing users from opening multiple tabs and injecting cloned Socket IDs into the same private lobby required complex session-tracking logic to ensure one user equaled one active connection.
- **Multiplayer Elo Calculations:** Adapting the standard Elo formula (which is strictly designed for 1v1 chess matches) to handle dynamic 3-to-10 player matches. We had to engineer a custom algorithm that treated a multi-player match as a series of simultaneous 1v1 comparisons based on final placement vectors.
- **React Rendering Bottlenecks:** Rendering a dynamic 30x30 hex-grid (900 individual cells) caused severe frame-rate drops initially. We had to heavily optimize the frontend

using `useMemo` and `useCallback` hooks to ensure only the specific grid cells affected by a move were re-rendered.

7.2 The "Next Step": Top 3 Features to Enhance the Project

- **Feature 1: Asynchronous Match Replay Engine**
 - *Description:* Appending all validated turn vectors (movements, territory captures, resource expenditures) to a MongoDB document array. This would allow players to fetch a match log post-game and scrub through the timeline chronologically to analyze their strategic mistakes.
 - *Effort Estimate:* ~**20 Hours** (Requires designing an efficient compression schema for the vectors to prevent DB bloat, and a frontend playback UI).
- **Feature 2: Redis Pub/Sub Integration for Horizontal Scaling**
 - *Description:* Currently, Mitrrio relies on a single Node.js instance to hold the Socket.io connections in memory. To scale seamlessly for thousands of users, we would implement a Redis adapter. This allows the socket events to be published and subscribed across multiple load-balanced server instances.
 - *Effort Estimate:* ~**15 Hours** (Requires migrating the in-memory `activeGames` map to a centralized Redis cache and refactoring socket emitters).
- **Feature 3: Custom Map Editor & Dynamic Rulesets**
 - *Description:* Giving Private Lobby hosts the ability to manipulate the procedural generation parameters before a match starts. Hosts could manually draw hex layouts, adjust the density of resources, or set custom turn-time limits.
 - *Effort Estimate:* ~**20 Hours** (Requires building a secondary "Canvas Editor" UI and expanding the backend validation engine to accept dynamic geometric constraints).

7.3 Hindsight: What We Would Do Differently & Tips for Future Teams

What We Would Do Differently:

- **Enforce Strict Type Safety from Day 1:** Passing untyped JSON payloads between the React frontend and the Socket.io backend led to several silent bugs where a misspelled variable (e.g., `userId` vs `user_id`) crashed the game loop. If starting over, we would enforce TypeScript interfaces shared across both environments to guarantee contract adherence.

Tips for Future Teams:

- **Design for Disconnects First.** Never assume a user will gracefully exit your application. Build your fault-tolerance architecture (reconnection logic, session caching, and cleanup intervals) before you write a single line of game logic.
- **Decouple Network Logic from UI Renders.** Never place socket emission or listening logic directly inside a standard React component's render cycle or basic `useEffect`.

Abstract all networking into custom hooks or a centralized context provider to prevent catastrophic memory leaks and infinite re-render loops.

- **Validate External Environment Constraints Early.** Do not assume that because a feature (like email SMTP or file writing) works on localhost, it will work in the cloud. Test your deployment environment constraints in Week 1 to avoid massive architectural pivots in Week 10.
- **The Server is the Absolute Source of Truth.** Never trust the client. Every coordinate click, resource spend, and state transition must be treated as hostile until mathematically validated by the backend engine.

7.4 AI Integration: Current Applications & Future Potential

How AI Was Utilized During Development:

- **Multi-LLM Game Engine Architecture:** We actively integrated Generative AI tools (like ChatGPT and Claude) as pair-programming assistants to accelerate the development of our core game engine. Drawing directly from our course seminar research, we experimented with a multi-LLM framework where different models were prompted to converse, debate, and validate each other's output. This method was highly effective for architecting complex backend logic, such as the spatial grid validation math, the multiplayer Elo calculation loops, and troubleshooting tricky `Socket.io` namespace memory leaks.

Where Else AI Could Have Helped (Future Enhancements):

- **Procedural Map Balancing:** Instead of manually tweaking the procedural generation noise algorithms to ensure fair spawn locations, we could have utilized an AI agent to simulate 10,000 map generations, evaluating and adjusting the resource distribution mathematically to guarantee absolute fairness for all spawn points.
- **Reinforcement Learning Bots (Playtesting):** Integrating a lightweight neural network to act as a "Bot Player." Training an AI to play Mitrrio against itself would have rapidly exposed dominant, game-breaking strategies or unit imbalances, allowing us to patch exploits much faster than manual human QA testing.

8. Team Contribution Statement

Feature / Module	Primary Developer	Helper / Co-Developer	Git History Context & Notes
Backend Architecture & Socket Lobby	Anishek	Ritika	Anishek built the foundational socket rooms and handled multi-tab edge cases (<i>"Fixed Room and multi tab users"</i>). Ritika stepped in to refactor the socket manager for better scalability (<i>"socket manager refactored"</i>).
Core Game Engine & Private Parties	Rohit	Anishek	Rohit authored the primary game logic and private party views (<i>"Game Created"</i> , <i>"Page with private party game created"</i>). Anishek assisted by hooking up the frontend rendering and game states (<i>"New game working"</i>).
Multiplayer Elo & Matchmaking	Anishek	Ritika	Anishek engineered the math and logic for the multiplayer Elo ranking system (<i>"working game and elo"</i>), while Ritika helped prep the codebase and UI for the Elo integration (<i>"Before elo"</i>).
Spectator "Shadow" Mode	Ritika	Anishek	Ritika took ownership of the spectator view and isolated chat features (<i>"sort of working spectator except edge cases"</i>). Anishek helped debug the final socket room edge cases to get it working perfectly.

Admin Dashboard & UI Polish	Ritika	Rohit	Ritika designed and integrated the secure admin dashboard to monitor system telemetry (<i>"admin dashboard created"</i>), as well as adding dynamic player colors to the lobby (<i>"party color added"</i>).
Authentication & Email Integration	Ritika	Anishek	Ritika authored the initial backend server code and email handlers. When cloud providers blocked our mailing ports, both Ritika and Anishek worked together for several days to patch it (<i>"Email fix"</i> , <i>"fixing mailing error"</i>).
Deployment, DB Hooks & Final Polish	Anishek	Rohit	Anishek handled the final production bottlenecks, specifically fixing Mongoose async database hooks and prepping the final deployment build (<i>"Fix pre-save hook: use async without next"</i> , <i>"Final working version ready for deployment"</i>). Rohit assisted with final deployment checks (<i>"Ready for deploying"</i>).

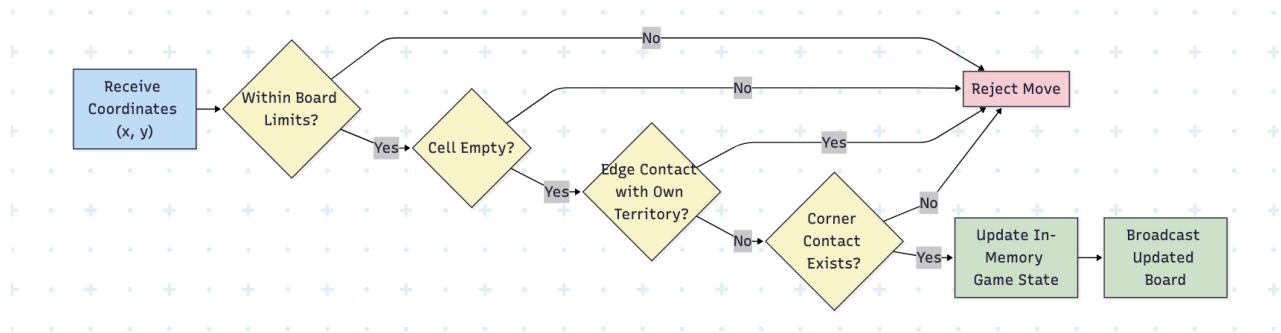
9. Appendix

The appendix provides supplementary technical material, sample data records, and qualitative reflections that offer deeper insight into the design, implementation, and evaluation of Mitrrio.

A.1 Extended Logic Flowcharts

A.1.1 Spatial Validation Algorithm (Corner-Touch Logic)

The spatial validation algorithm is one of the most critical parts of the Mitrrio game engine. Unlike conventional grid-based games, Mitrrio enforces a strict placement rule in which a player's new piece must connect only through corners with their existing territory and must not share any flat edge with it. This requires a sequence of validation checks before a move can be accepted.



The validation process follows these steps:

- 1. Input Reception:**

The server receives a set of $(x,y)(x, y)(x,y)$ coordinates representing the intended placement of the selected piece.

- 2. Boundary and Overlap Check:**

Each coordinate is verified to ensure that it lies within the valid board dimensions and that the target cell is currently unoccupied.

- 3. Edge-Block Check:**

The system checks the immediate neighboring cells in the north, south, east, and west directions. If any of these cells already belong to the current player, the move is rejected.

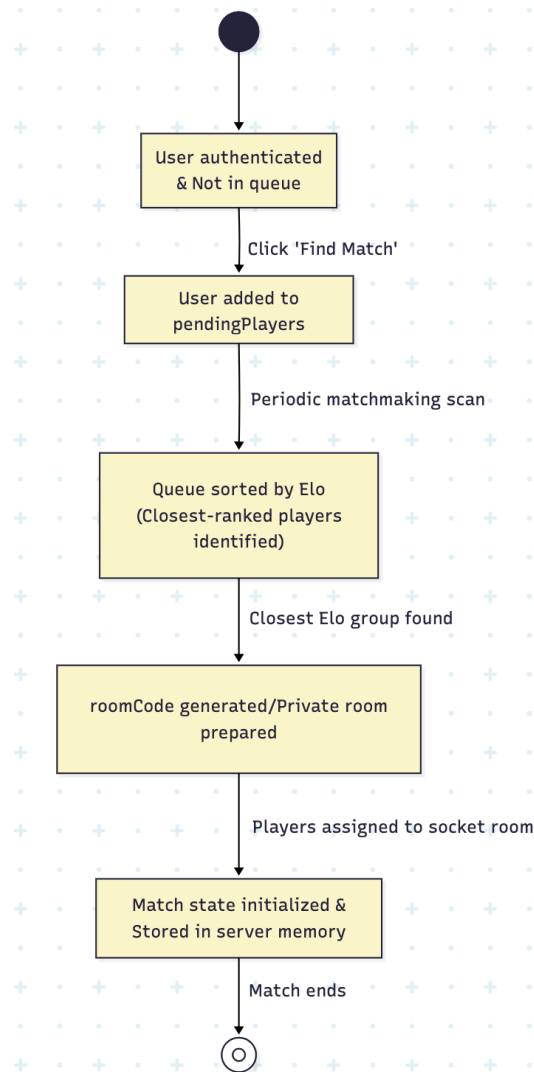
- 4. Corner-Touch Check:**

The diagonal neighboring cells in the northeast, northwest, southeast, and southwest directions are examined. At least one cell of the new piece must touch the player's existing territory diagonally.

- 5. State Broadcast:**

If all checks are satisfied, the in-memory activeGames state is updated and the new board state is broadcast to all connected sockets in the room.

A.1.2 Matchmaking State Machine



To ensure a smooth and responsive multiplayer experience, the matchmaking service can be modeled as a finite state machine with the following states:

- **Idle:**
The user is authenticated but is not currently in the matchmaking queue.
- **Queuing:**
The user is added to the pendingPlayers queue, where the system continuously checks for compatible players.
- **Initialization:**
Once the required player threshold (2--10 players) is met, a unique roomId is generated and the selected players are moved from the global queue into a private socket room.
- **Active:**
The match begins, and the live game state is maintained in server memory for efficient real-time updates.

A.2 Sample Results and Database Records

A.2.1 Sample Match Document (MongoDB)

The following sample document illustrates how a completed match is stored for administrative review and user match history.

```
{
  "_id": "64f1a2b3c4d5e6f7",
  "roomCode": "MTR-8821",
  "status": "completed",
  "boardSize": 20,
  "players": [
    { "username": "Player1", "score": 85, "rank": 1, "eloGained": 24 },
    { "username": "Player2", "score": 72, "rank": 2, "eloGained": 12 },
    { "username": "Player3", "score": 45, "rank": 3, "eloGained": -18 }
  ],
  "finalBoard": [[null, "blue", "blue"], ["red", "red", null]],
  "createdAt": "2026-04-10T14:30:00Z"
}
```

A.2.2 Admin Dashboard Telemetry

During final testing, the admin dashboard successfully tracked the following runtime metrics:

- **Peak Concurrency:** 85 active WebSocket connections
- **Average Game Duration:** 12.5 minutes for a 4-player session
- **Validation Latency:** Average place_piece validation time of 14 ms, which remained well below the 100 ms target threshold

A.3 Personal Reflections

A.3.1 Technical Resilience: The SMTP Pivot

One of the most valuable lessons during the development of Mitrio was the importance of architectural flexibility. During deployment on Render, verification emails failed because outbound SMTP traffic over Port 587 was blocked. Instead of continuing with a traditional SMTP-based solution, the email verification module was redesigned to use the Brevo REST API. This shift demonstrated that in cloud environments, API-first approaches are often more reliable and deployment-friendly than legacy mail protocols.

A.3.2 The Challenge of Multiplayer Concurrency

Managing up to 10 simultaneous players within the same game room introduced significant concurrency challenges. In early testing, race conditions occurred when multiple players attempted to place pieces at nearly the same instant. To solve this, a strict server-side event queue

was introduced so that socket events could be processed sequentially and the shared board state could be updated atomically. This greatly improved consistency and deepened our understanding of event-driven concurrency in Node.js.

A.3.3 Collaborative Vision

The development of Mitrrio evolved from a basic MERN stack foundation into a fully functional real-time multiplayer platform through a shared team vision. One of the most rewarding aspects of the project was balancing engaging gameplay and user interface design with the rigor of spatial validation logic and real-time synchronization. Successfully completing a 10-player match with correct score calculation and Elo updates was a strong validation of the system design.

A.4 User Manual Snippets

- **Getting Started:** Register an account and verify your email before logging in.
- **Matchmaking:** Use the **Find Match** button to join a skill-balanced multiplayer queue.
- **Controls:** Select a piece, use the **R** key to rotate it before placement.
- **Core Rule:** A newly placed piece must touch your existing territory only at the corners. Edge-to-edge contact with your own color is not allowed.